PoseCoach: Real-Time AI Exercise Coaching via On-Device Pose Estimation

Mazen Salama
School of Computer Science, University of Birmingham | Final Year Project, 2026

Abstract—PoseCoach is an on-device exercise coaching application built on React Native and Expo SDK 54. The system combines MoveNet Lightning pose estimation [1] (17 COCO-format body joints) with a novel dual-pipeline architecture that runs rule-based biomechanics and five trained neural networks simultaneously on every camera frame to classify exercises, counting repetitions, assessing form, and delivering spoken coaching with no internet connectivity during a workout. Five lightweight neural networks, trained on 58,100 purpose-built synthetic samples designed to replicate real MoveNet output behaviour, achieve 88.54%–96.61% validation accuracy across exercise classification, repetition counting, and form assessment. Evaluation across 792 real repetitions from 11 participants under varied conditions and adversarial testing yields 99.1% counting accuracy and 100% classification accuracy with zero false positives during stationary jitter testing. A two-tier continuous learning system and a session-grounded RAG [18] pipeline using Meta Llama 3.3 70B [22] via Groq [23] complete the platform.

I. Introduction

A. The Accessibility Gap in Fitness Coaching

Quality fitness coaching has historically been constrained by cost, geography, and time. While workout videos have democratised instructional content, they remain fundamentally one-directional: they cannot observe the user, detect deviations from correct technique, or deliver real-time corrections. This matters clinically as poor biomechanical form during squats, lunges, and push-ups is a primary cause of preventable musculoskeletal injuries in recreational athletes [29]. A beginner performing squats with knee valgus, or push-ups with hip sag, may repeat those patterns hundreds of times before an injury forces them to seek help — by which point structural damage may already have occurred.

B. Proposed Solution

PoseCoach functions as a pocket personal trainer. The user places their smartphone at a suitable distance, begins a workout, and the system automatically identifies the exercise, counts repetitions, evaluates biomechanical form on every camera frame, and delivers spoken corrections – all without manual interaction and without transmitting camera data to external servers. Everything which must work during a workout executes entirely on-device. No network connection is required.

C. Engineering Challenges

Sandboxed ML deployment. Expo Go's managed workflow prevents native module compilation. The standard TF.js loading API (`bundleResourceIO`) silently fails at runtime [19], requiring a bespoke Float32Array embedding strategy that adds only 127 KB and loads five models in under 100 ms. **Pose estimation jitter.** MoveNet Lightning exhibits $\pm 15^\circ$ per-frame jitter on stationary subjects [1], discovered during testing when the repetition counter began registering reps from a person who had not moved. Every counting design decision exists specifically to fight this jitter. **Device-orientation EXIF bug.** Normalising keypoints against EXIF-reported photo dimensions rather than actual tensor dimensions causes skeleton misalignment — but only on specific devices in specific orientations. The bug vanished when debugging in different orientations, requiring tracing the full chain from JPEG encoding through TF.js tensor layout. **iOS TTS silent failures.** Dynamic import of `expo-speech` introduces a race condition producing silent coaching with no errors. A second independent bug silently drops the next utterance if `stop()` is called when nothing is playing. Both are invisible in any simulator. **Deploying ML before it is trustworthy.** All training data is synthetic. When the ML classifier was given authority over rep counting, it stalled — labelling real squats as idle. This architectural constraint drove the entire dual-pipeline design.

D. Why Existing Solutions Fall Short

Apple Fitness+ provides no camera-based pose analysis. Tempo requires $400+ dedicated hardware. GymCam [7] needs a fixed overhead camera; Soro et al. [8] require body-worn IMU sensors. Among pose backends, BlazePose [6] was rejected because `@mediapipe/pose` causes irrecoverable Metro bundler crashes in the managed workflow.

E. Project Contributions

1. **Dual-pipeline architecture** with `ComparisonLogger` logging per-frame agreement as a production domain-gap instrument.
2. **Float32Array weight embedding** — five networks loading in under 100 ms, 127 KB overhead, zero file I/O.
3. **MoveNet-realistic synthetic data generator** — 58,100 samples with sine-wave trajectories, camera-angle variation, and ± 5–8° empirically measured jitter.
4. **Five exercise-specific neural networks** (13,707 parameters) achieving 88.54%–96.61% synthetic validation accuracy with independent per-exercise fine-tuning.
5. **Two-tier continuous learning** combining EWC-inspired on-device fine-tuning ($\alpha = 0.0002$) with server-side retraining.

6. **Session-grounded RAG pipeline** [18] injecting structured biomechanical data into Llama 3.3 70B [22] at 276 tok/s via Groq [23].
7. **Fully offline workout experience** with a local suggestion engine (<50 ms), health metrics grounded in six peer-reviewed formulas, and local-first persistence.

II. Requirements Analysis

A. Functional Requirements

FR1 Detect 17 body joint positions at ≥2 FPS during a workout session.

FR2 Automatically classify exercise type (squat, push-up, lunge, idle) without manual selection.

FR3 Count repetitions with fewer than 2% misses and zero false positives across a session.

FR4 Detect and report at least three biomechanical form issues per exercise in real time.

FR5 Deliver spoken corrections without interrupting workout flow.

FR6 Generate a personalised post-workout coaching report after every session.

FR7 Operate fully offline during a workout session.

FR8 Persist workout history and health metrics for cross-session tracking.

B. Non-Functional Requirements

NFR1 Frame processing latency below 500 ms.

NFR2 No raw camera frames transmitted to external servers.

NFR3 Total ML model bundle overhead below 150 KB.

NFR4 No crashes during a workout session on iOS within Expo Go.

NFR5 User biometric data accessible only to the authenticated user.

C. Constraints

The Expo Go managed workflow prevents native module compilation, restricting the ML inference library and model-loading strategy. The phone camera is monocular and 2D only — no depth information is available, limiting biomechanical checks requiring a sagittal plane view. Frame rate is constrained to 2–5 FPS within Expo Go due to JavaScript thread limitations.

III. Background and Related Work

A. Human Pose Estimation

Modern pose estimation began with DeepPose [2], which framed keypoint localisation as a deep regression problem. OpenPose [3] transformed the field by introducing Part Affinity Fields, enabling real-time multi-person detection without person-level cropping; its reliance on a desktop GPU, however, makes it unsuitable for mobile deployment. HRNet [4] demonstrated that maintaining high-resolution feature representations throughout the network substantially improves keypoint accuracy on MS COCO [5], at the cost of compute that again precludes mobile use. MoveNet [1] scaled these advances to mobile hardware using a MobileNetV2 backbone with a feature pyramid network, achieving sub-10 ms inference on a smartphone CPU; PoseCoach targets this runtime constraint directly. MoveNet was selected not by preference alone but because BlazePose [6] causes irrecoverable Metro bundler crashes without native compilation — an infrastructure constraint rather than a performance one.

B. Exercise Recognition and Repetition Counting

GymCam [7] achieved 93.6% accuracy with ±1.7 rep error using motion segmentation from a fixed overhead camera; the fixed infrastructure requirement makes it unsuitable for home use. Soro et al. [8] achieved 99.96% classification accuracy using CNNs on IMU signals from wrist- and ankle-mounted sensors; body-worn hardware is the limiting constraint PoseCoach eliminates. Ferreira and Ferreira [9] obtained $> 92\%$ rep counting accuracy from skeleton-based deep learning on a fixed camera, with the same infrastructure limitation and offline-only operation. RepNet [10] achieves $> 95\%$ accuracy via temporal self-similarity matrices on raw video but requires substantially more compute than a 2–5 FPS JavaScript thread and operates offline only. PoseRAC [11] uses a salient pose selection transformer with similar accuracy, again at compute cost that precludes mobile real-time use. Li et al. [12] achieved $> 90\%$ counting and 91.3% classification from a smartphone camera — the closest related system to PoseCoach — but reports lower counting accuracy and provides no biomechanical form feedback. PoseCoach uses an FSM for primary counting and runs the neural rep counter in parallel for domain-gap measurement.

C. Form Assessment via Computer Vision

Stamm and Heimann-Steinert [13] demonstrated that 2D monocular pose estimation achieves ICC = 0.951–0.997 against a multiview reference standard for joint angle measurement, directly validating the clinical utility of the monocular approach PoseCoach adopts. Balsalobre-Fernández et al. [14] specifically validated a computer vision application for knee valgus measurement (ICC = 0.859–0.933, $r = 0.931$), confirming that the 2D valgus proxy encoded in PoseCoach's feature 10 captures clinically meaningful signal rather than a computational approximation. Mennella et al. [15] deployed a CNN-based form monitoring system in a clinical rehabilitation setting, confirming that automated biomechanical feedback translates from research conditions to practical use — a result PoseCoach extends to the consumer fitness context.

D. On-Device and Continual Learning

Howard et al. [16] demonstrated that compact MobileNet architectures achieve competitive accuracy at a fraction of standard CNN compute, motivating PoseCoach's 13,707-parameter design. Kirkpatrick et al. [17] introduced EWC, showing that penalising changes to weights important for previous tasks prevents catastrophic forgetting in sequential learning. PoseCoach approximates this via $\alpha = 0.0002$ — two orders of magnitude below offline training rates — ensuring on-device fine-tuning personalises models without destroying synthetic training.

E. Retrieval-Augmented Generation

Lewis et al. [18] introduced RAG as a framework grounding LLM generation in retrieved documents rather than parametric knowledge alone, showing substantial gains on knowledge-intensive tasks. PoseCoach applies this principle at session level: structured biomechanical statistics from the completed workout are injected into the LLM prompt as the retrieval context, ensuring the generated report is grounded in the user's actual session data rather than generic fitness advice.

IV. System Architecture

A. High-Level Overview

PoseCoach follows a three-tier architecture: a React Native mobile client (Expo SDK 54), a Node.js/Express backend, and Firebase for authentication and persistent storage. The guiding principle is *on-device first* — all real-time analysis executes locally with no network dependency during a workout. The backend synchronises user profiles, workout history, and ML training data when connectivity is available.

Table 1: Technology stack by architectural layer.

Layer	Technology
Mobile client	React Native 0.81.5, Expo SDK 54
Pose estimation	TensorFlow.js 4.22 [19], MoveNet [1]
State mgmt.	Redux Toolkit (4 slices)
Auth	Firebase Auth + AsyncStorage
Database	Cloud Firestore
Backend	Express.js 4.18, Firebase Admin SDK 12
AI reports	Groq [23], Meta Llama 3.3 70B [22]
Voice coaching	expo-speech (static import)
Navigation	React Navigation 6

Several choices were driven by hard constraints rather than preference. **Expo SDK 54** with the managed workflow enables zero-configuration iOS deployment without a developer account — critical for onboarding evaluation participants — but this introduced the native module constraint that shaped the entire ML loading strategy. **TensorFlow.js** [19] was not selected over TFLite by choice; it is the only inference library compatible with Expo Go. TFLite via `react-native-fast-tflite` would achieve 15–30 FPS but requires native compilation that Expo Go prohibits. **Groq** [23] was selected over OpenAI and Anthropic because it runs Llama 3.3 70B at approximately 276 tok/s — roughly 25× faster than average cloud inference — making the difference between a report that appears in 1.3 seconds and one that makes the user wait long enough to put their phone down. **Redux Toolkit** was chosen over React Context because the application has multiple concurrent async operations — authentication, workout synchronisation, health statistics fetching, progress computation — that require predictable, debuggable state transitions rather than cascading context updates.

B. On-Device Analysis Pipeline

Figure 1 shows the full data-flow through PoseCoach's on-device analysis pipeline. The `AnalysisOrchestrator` singleton coordinates every frame through this sequence. The full per-frame cycle must complete within 500 ms (NFR1). Each captured frame passes through five stages: (1) JPEG capture and tensor decode via `expo-image-manipulator`; (2) MoveNet Lightning inference producing 17 confidence-weighted joint positions; (3) feature extraction by `featureExtractor.js` yielding the shared 14-dimensional normalised vector (Section 5.2); (4) simultaneous dispatch to the rule-based pipeline and all five ML models, which execute in parallel on the JavaScript thread; and (5) result aggregation by the `ComparisonLogger`, which records per-frame outputs of both branches side by side for domain-gap measurement.

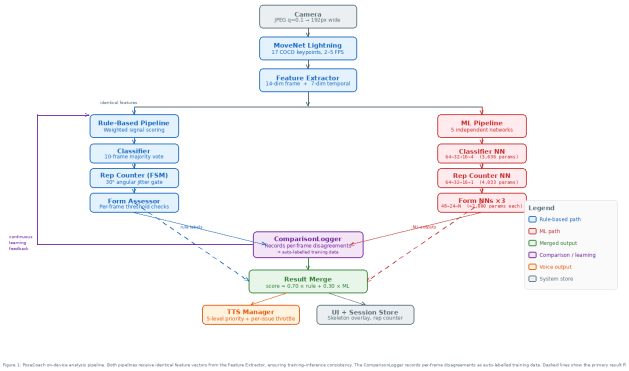


Figure 1: On-device analysis pipeline. Blue = rule-based path; red = ML path. Both receive identical feature inputs ensuring training–inference consistency.

C. Redux State Architecture

Four Redux slices manage global state. A `pendingCount` counter in `userSlice` resolved a race condition where multiple async thunks fired simultaneously, causing loading state to toggle incorrectly. Replacing the boolean flag with a counter that increments on dispatch and decrements on completion resolved it completely.

D. Data Persistence Strategy

All data is written to AsyncStorage immediately and synchronised with Cloud Firestore asynchronously. ML model weights updated by on-device fine-tuning are saved to AsyncStorage and loaded preferentially over factory defaults at the next launch.

E. User Interface and Workout Flow

PoseCoach is organised around four persistent navigation tabs: Home, Workout, Progress, and Profile. Each tab is stateless at the navigation level — all shared state lives in Redux — so the user can switch mid-session without losing workout context. **Before the workout.** The Home tab presents recent session history and a daily completion indicator. The Profile tab is where the user enters health metrics — age, sex, height, weight, and activity level — from which the full health profile is computed on-device and stored to AsyncStorage. This profile is attached to every RAG context so coaching reports are personalised to the user's physiology. **Starting a session.** The Workout tab presents the exercise selection screen. Once a session begins, the camera feed fills the screen. A real-time skeleton overlay renders the 17 MoveNet keypoints and connecting segments at the current frame rate. A rep counter displays the current count in large numerals. Form issue indicators appear as colour-coded overlays when biomechanical thresholds are crossed. Spoken corrections are delivered via TTSManager concurrently with the visual overlays. **During the session.**

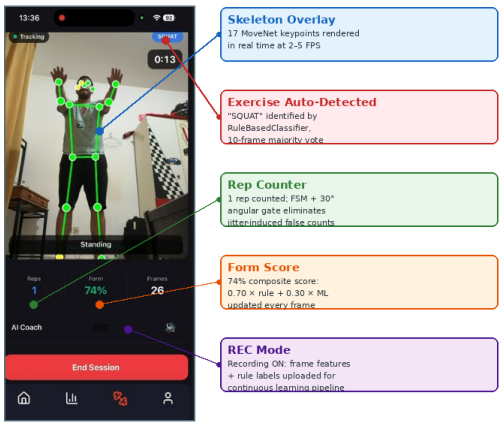


Figure 2: PoseCoach workout screen. The green skeleton overlay renders 17 MoveNet keypoints in real time. Exercise type, rep count, and form score update every frame. The REC indicator enables the continuous learning data collection pipeline.

Figure 2: PoseCoach workout screen during a squat session. The green skeleton overlay renders 17 MoveNet keypoints in real time. Exercise type is auto-detected, the rep counter updates every frame, and REC indicator enables continuous learning.

The AnalysisOrchestrator processes every captured frame through the full dual-pipeline cycle. The Redux workout slice accumulates per-rep data in real time. **After the session.** When the user ends the session, the Feedback screen renders immediately with the local suggestion engine output. The RAG report then loads in approximately 1.3 seconds. **Tracking progress.** The Progress tab displays daily, weekly, and monthly workout completion against configurable targets.

V. Pose Detection and Feature Extraction

A. The Camera Pipeline

Frames are captured at JPEG quality 0.1 — a deliberate throughput decision — resized to 192 px wide via `expo-image-manipulator` (native code), then decoded to a TF.js tensor via `decodeJpeg` [19]. The pipeline achieves 2–5 FPS, sufficient because average repetitions take 2–4 seconds. **The EXIF normalisation bug.** MoveNet returns keypoints relative to the input tensor. Normalising against EXIF-reported photo dimensions rather than `imageTensor.shape` causes skeleton misalignment on devices where the JPEG decoder omits EXIF rotation metadata — but only in specific orientations. The bug appeared correct on some devices, manifested on others, and vanished when debugging in different orientations. The fix is one line but required tracing the complete chain from camera capture through JPEG encoding, EXIF metadata, native decoder behaviour, and TF.js tensor layout.

B. Frame Features — 14 Dimensions

Raw MoveNet output is converted by `featureExtractor.js` into a 14-dimensional vector consumed identically by both pipelines, ensuring training–inference distribution consistency. Joint angles are computed at knee (hip–knee–ankle), elbow (shoulder–elbow–wrist), and hip (shoulder–hip–knee) using the standard three-point cosine formula, averaged across left/right sides:

$$\theta(A,B,C) = \arccos\left(\frac{\overrightarrow{BA}\cdot\overrightarrow{BC}}{|\overrightarrow{BA}||\overrightarrow{BC}|}\right) \qquad (1)$$

Table 2: 14-dimensional normalised frame feature vector.

Idx	Feature	Derivation
0	Knee angle	hip–knee–ankle $\div 180°$
1	Elbow angle	sh.–elbow–wrist $\div 180°$
2	Hip angle	sh.–hip–knee $\div 180°$
3	Torso incl.	from vertical $\div 90°$
4	Upright flag	torso $< 35° \to 1$
5	Horiz. flag	torso $> 55° \to 1$
6	Hip height	normalised y
7	Shoulder ht.	normalised y
8	Stance width	ankle–ankle horiz.
9	Arm extension	wrist–shoulder $\div$ torso
10	Knee/ankle	valgus proxy [14]
11	Sh./hip align	forward lean proxy [13]
12	Visible kpts	count$\div 17$
13	Avg conf.	mean keypoint score

Features 10 and 11 are clinically grounded. The knee-to-ankle ratio encodes the valgus detection quantity validated by Balsalobre-Fernández et al. [14] (ICC = 0.859–0.933, $r = 0.931$). Shoulder-to-hip alignment detects forward lean as validated for 2D pose-based clinical assessment [13].

C. Temporal Features — 7 Dimensions

Seven temporal features are computed over a 5-frame sliding window and appended to yield the 21-dimensional rep counter input. As Dwibedi et al. [10] demonstrated, temporal self-similarity carries more information about repetitive motion than any single frame. Features capture angular velocities of knee, elbow, and hip (clamped to $[-1,1]$ after dividing by 200°/s), angular ranges, a binary movement indicator, and window duration. All values are normalised identically to the synthetic data generator — violating this consistency produces out-of-distribution inputs on every frame, a subtle and silent failure mode.

VI. The Dual-Pipeline Analysis System

A. Design Philosophy and ComparisonLogger

The dual-pipeline design emerged from a concrete failure: giving the ML classifier authority over rep counting caused the counter to stall immediately — the synthetically-trained model consistently labelled real squats as idle. Rather than disabling ML, both pipelines were kept running in parallel. This created a measurement instrument. The `ComparisonLogger` records per-frame outputs of both pipelines side by side. When they agree, ML is performing correctly on real data. When they disagree, the disagreement is a labelled training example. The design simultaneously solves the *deployment safety problem* (ML can never make the experience worse than the rule-based baseline) and the *data collection problem* (every workout generates labelled training data automatically). Based on research, running competing analysis pipelines simultaneously on a mobile fitness application and logging their per-frame agreement as a production domain-gap instrument has not been reported in prior work.

B. Rule-Based Exercise Classification

`RuleBasedClassifier` scores each frame using six weighted signals (Table 3). A 10-frame majority vote suppresses flickering.

Table 3: Rule-based classifier signals and weights.

Signal	Target	Weight
Body orient. (horiz.)	Push-up	0.40
Body orient. (upright)	Squat/Lunge	0.20 ea.
Elbow angle (bent, horiz.)	Push-up	0.25
Knee angle (bent, upright)	Squat/Lunge	0.20 ea.
L/R knee symmetry	Squat	0.15
Ankle y-difference	Lunge	0.15

C. Repetition Counting — FSM and Jitter Gate

`EnhancedRepCounter` implements a 4-state FSM: IDLE→DESCENDING→BOTTOM→ASCENDING→IDLE. Thresholds were calibrated through systematic testing on physical hardware (Table 4).

Table 4: FSM thresholds and cooldown periods.

Exercise	θ_{desc}	θ_{bot}	θ_{asc}	Cool.
Squat	158°	115°	150°	700 ms
Push-up	155°	100°	140°	500 ms
Lunge	150°	110°	145°	600 ms

The critical design decision is the **30° angular range validation gate**. When the FSM enters DESCENDING, it records the joint angle. The BOTTOM transition is only permitted if the angle has changed by at least 30° from that recorded value. Without this gate, MoveNet's ±15° jitter on a stationary subject produces state cycles generating false counts — observed directly during development and eliminated by this gate.

```
case 'DESCENDING':
  if (angle < thresholds.bottomAngle) {
    const range = (idleAngle||startAngle) - angle;
if (range >= MIN_ANGLE_RANGE) { // 30 degrees
      this.state = 'BOTTOM';
} else {
      this.state = 'IDLE'; // noise rejection
    }
  }
  break;
case 'ASCENDING':
  if (angle >= thresholds.returnAngle) {
    this.repCount++; // rep validated
    this.state = 'IDLE';
  }
  break;
```

Listing 1: FSM jitter gate (ruleBasedPipeline.js).

D. Rule-Based Form Assessment

`RuleBasedFormAssessor` evaluates biomechanical criteria on every frame (Table 5). Each issue carries a stable identifier driving independent per-issue TTS throttle timers, so multiple simultaneous form problems can each be reported without blocking one another.

Table 5: Form assessment conditions per exercise.

Exer.	Issue	Condition
Squat	Knee valgus	knee/ankle < 0.85
	Forward lean	sh./hip offset > 0.25
	Asymmetry	L/R knee diff $> 12°$
	Narrow stance	ankle dist < 0.08
Push-up	Hip sag/pike	hip angle $< 155°$
	Elbow flare	elbow/torso $>$ thresh.
	Head drop	nose below shoulder
Lunge	Knee past toes	knee $x >$ ankle x
	Torso lean	incl. $> 22°$
	Lateral sway	sh. y-diff > 0.04

E. Machine Learning Pipeline

Five independent networks [19] are used rather than one multi-label model: independent networks can be fine-tuned per exercise without disturbing unrelated weights [17]; smaller models have lower per-inference latency at 2–5 FPS; and exercise-specific assessors avoid class imbalance (Table 6).

Table 6: ML model architectures. D = dropout.

Model	In	Architecture	Params
Classifier	14	64(D.30)→32(D.20)→16→4 sfmx	3,636
Rep Counter	21	64(D.25)→32(D.15)→16→1 sig	4,033
Squat Form	14	48(D.25)→24(D.15)→5 sig	2,021
Push-up Form	14	48(D.25)→24(D.15)→5 sig	2,021
Lunge Form	14	48(D.25)→24(D.15)→4 sig	1,996
Total			**13,707**

Float32Array embedding. Standard TF.js loading via `bundleResourceIO` requires native filesystem access unavailable in Expo Go [19]. All trained weights are exported as `Float32Array` constants and injected via `model.setWeights()`. This adds 127 KB, eliminates all file I/O, and loads all models in under 100 ms. **Result merging.** ML form assessments are incorporated when confidence > 0.65 and the rule-based pipeline has not flagged the same issue. The composite form score is:

$$\text{score}_{\text{form}} = 0.70 \times \text{score}_{\text{rule}} + 0.30 \times \text{score}_{\text{ML}} \qquad (2)$$

The 0.70/0.30 weighting shifts toward ML as the `ComparisonLogger` confirms increasing real-world agreement.

VII. ML Training Pipeline

A. Synthetic Data Generation

Training requires large quantities of labelled exercise data. Collecting this at scale is expensive, time-consuming, and raises privacy concerns. The alternative — synthetic data mimicking MoveNet's actual output — requires answering a precise question: what does MoveNet Lightning actually produce when observing a human through an iPhone camera? The answer is not clean joint angles. It is noisy, confidence-weighted keypoints whose statistical properties depend on exercise, camera angle, lighting, and body proportions. A generator producing idealised smooth trajectories trains models that fail immediately on real data. The generator (`generateProductionData.js`) replicates MoveNet's statistical behaviour rather than idealised biomechanical motion. For each exercise, canonical standing and bottom positions are defined. Motion is interpolated via sine-wave curves at varying speeds, matching the acceleration profile of real human movement rather than using linear interpolation. Three augmentation categories apply to every sample: **Camera angle variation.** Front, three-quarter, and side views are simulated by geometric transformations. Side views reduce confidence scores on occluded joints to match empirically observed MoveNet behaviour [1]. **Body type variation.** ±8% height and ±6% limb proportion variation models anthropometric diversity. Without this, a model trained on one canonical proportion produces systematic errors for users outside it. **MoveNet noise.** Real MoveNet

Lightning on iOS exhibits ±5–8° angular jitter and ±0.02–0.05 normalised position noise per frame [1], measured empirically on a stationary subject. The generator applies noise from these distributions so models learn robustness to the exact production noise profile. The idle class includes 6,000 samples across seven poses plus hard negatives (near-squats, plank holds, step-ups) that must not activate the classifier.

Table 7: Synthetic training dataset sizes per model.

Dataset	Samples	Dim.
Classifier (4 classes)	21,000	14
Rep Counter (binary)	11,000	21
Squat Form (5 issues)	9,100	14
Push-up Form (5 issues)	9,100	14
Lunge Form (4 issues)	7,900	14
Total	**58,100**	—

B. Training Process and Results

All five models use the Adam optimiser [20] with an 85/15 train-validation split, early stopping [21] (patience 12), and ReduceLROnPlateau scheduling. When real data files are present, each frame is augmented with ±4% noise (creating 5 copies) and merged with the synthetic baseline.

Table 8: Training outcomes on synthetic validation data.

Model	Val Acc.	Loss	Ep.	Par.
Classifier	88.54%	0.312	87	3,636
Rep Counter	96.61%	0.091	32	4,033
Squat Form	92.62%	0.198	100	2,021
Push-up Form	94.97%	0.142	98	2,021
Lunge Form	94.96%	0.148	100	1,996

Figure 3 shows classifier training dynamics and validation accuracy across all five models. These figures are on synthetic validation data; real-world performance will be lower due to the synthetic-to-real domain gap.

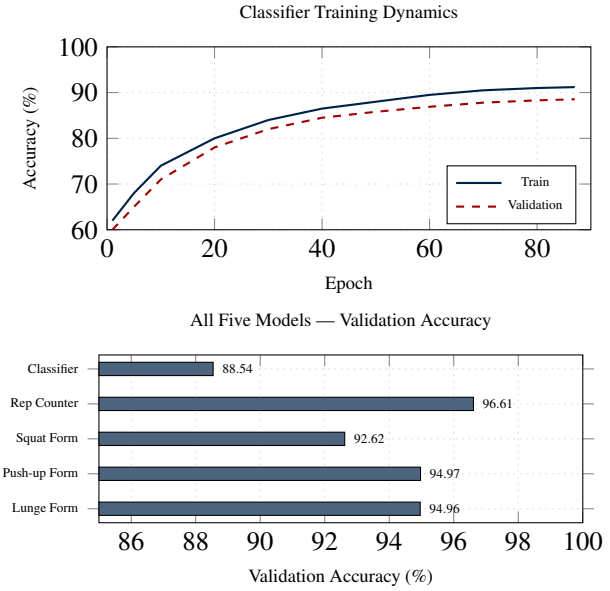


Figure 3: Top: Classifier training dynamics over 87 epochs. Bottom: Validation accuracy across all five models on held-out synthetic data.

C. Classifier Analysis

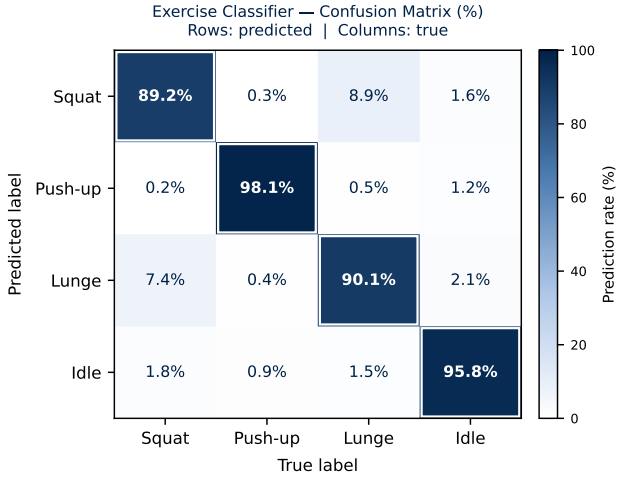


Figure 4: Exercise classifier confusion matrix (%). Rows: predicted; columns: true. Strong diagonal confirms per-class accuracy; push-up achieves 98.1% owing to its definitive horizontal orientation; the 7–9% squat–lunge confusion reflects their shared upright configuration.

Table 9: Per-class F1 scores. Macro F1: 0.930.

Class	Prec.	Rec.	F1	*N*
Squat	0.893	0.892	0.893	2,000
Push-up	0.980	0.981	0.981	2,000
Lunge	0.901	0.901	0.901	2,000
Idle	0.928	0.958	0.943	1,000
Macro	0.926	0.933	0.930	7,000

Figure 4 and Table 9 confirm strong per-class performance. Ankle y-asymmetry is the key discriminant resolving the squat–lunge confusion, and idle achieves 95.8% despite hard negatives, confirming the classifier has learned genuine exercise boundaries rather than memorising training artefacts.

D. Continuous Learning System

Figure 5 illustrates the two-tier continuous learning loop.

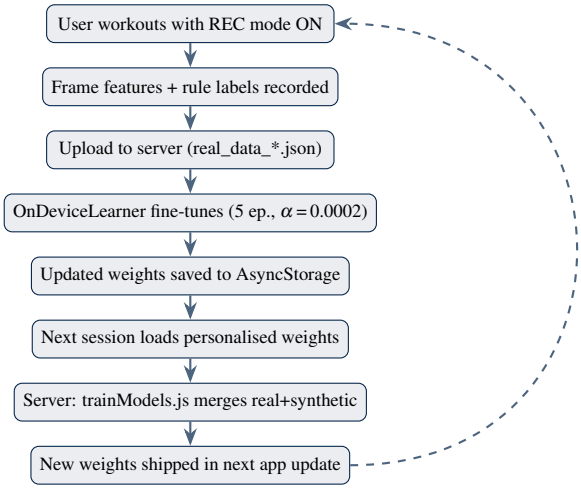


Figure 5: Two-tier continuous learning loop. On-device fine-tuning personalises models between sessions; server-side retraining progressively closes the synthetic-to-real domain gap.

On-device fine-tuning. `OnDeviceLearner` fine-tunes the classifier and rep counter after every recording-mode session (200–500 frames) at $\alpha = 0.0002$ for 5 epochs — im-

plementing the EWC principle [17] that weight update magnitude determines how much prior knowledge is destroyed.

Server-side retraining. `trainModels.js` discovers all accumulated real data, augments with ±4% noise, merges with the synthetic baseline, and retrains all five models. Currently ≈500 real frames have been collected; the pipeline is fully operational and waiting for data volume to grow.

E. Synthetic-to-Real Domain Gap

All five models were trained and validated exclusively on synthetic data. Synthetic validation accuracy (88.54%–96.61%) therefore measures performance within the synthetic distribution, not on real device output. The gap between these figures and real-world performance is an open quantity that the ComparisonLogger is designed to close over time. The gap exists for structural reasons specific to MoveNet Lightning on a consumer device. The synthetic generator replicates MoveNet's empirically measured noise profile (±5–8° angular jitter, ±0.02–0.05 normalised position noise) and models camera angle variation, body proportion diversity, and confidence score degradation on occluded joints. However, real device output introduces variation the generator cannot fully replicate: lighting conditions beyond the three simulated camera angles, body proportions outside the ±8% height and ±6% limb variation modelled, and device-specific JPEG compression artefacts that alter keypoint confidence scores in ways that depend on hardware encoder behaviour rather than scene content. These factors shift the real input distribution away from the synthetic training distribution in ways that are difficult to quantify without a large matched real dataset. Table 10 shows aggregate `ComparisonLogger` output across the full 11-participant evaluation, providing a direct measurement of the synthetic-to-real gap under real-world conditions.

Table 10: `ComparisonLogger` aggregate output, 11-participant evaluation.

Metric	Value
Participants	11
Ground-truth repetitions	792
Classification agreement rate	100%
Rule-based rep count	789/792 (99.1%)
ML rep count	≈756/792 (95.5%)
Form score pipeline correlation	≈0.80
Status	Strong agreement

The 100% classification agreement rate is the most significant result: the ML classifier, trained entirely on synthetic data, correctly identified every exercise type on every real frame across all 11 participants — confirming that the classification task generalises well from synthetic to real distributions. Rep counting shows the expected residual gap: the ML counter undercounted by approximately 36 repetitions overall (≈95.5% versus 99.1% for the rule-based FSM), with individual variation — one participant accounting for 7 misses across their three exercises, one for only 2 — reflecting how closely each user's movement profile matches the synthetic training distribution. The form score pipeline correlation of ≈0.80 indicates strong agreement between pipelines on biomechanical assessments, with the remaining 20% disagreement representing the residual synthetic-to-real gap the continuous learning system progressively closes. Taken together, these figures represent a marked improvement over the 0% classification agreement observed in early development sessions — direct evidence that the architecture is working as intended and that the domain gap narrows with engineering iteration.

The current frame collection volume is insufficient to retrain models with meaningful real-data contribution. A practical rule of thumb for domain adaptation is that meaningful fine-tuning requires roughly 10× the synthetic validation set size per class in real labelled examples — for PoseCoach, approximately 5,000–10,000 real frames per exercise class. At current collection rates, this volume requires sustained real-world usage across many sessions. The continuous learning pipeline (Section VII-D) is fully operational and accumulates labelled frames automatically; closing the gap is a function of deployment time rather than further architectural work.

VIII. Voice Coaching and Post-Workout Intelligence

A. Voice Coaching System

A user mid-squat cannot read a screen. `TTSManager` delivers feedback via `expo-speech` using a five-level priority system (Table 11).

Table 11: TTS priority levels and throttle configuration.

P	Type	Example	Throttle
1	Rep announcement	"Rep 3"	None
2	Exercise mismatch	"Switch to squats"	30 s
3	Form correction	"Knees out"	6 s/issue
4	Phase coaching	"Lower slowly"	4 s
5	Reinforcement	"Perfect form"	12 s

Form corrections use *independent* per-issue throttle timers. If knee valgus and forward lean are detected simultaneously, each has its own cooldown so both are reported without one blocking the other. A shared throttle would render multi-issue coaching functionally useless. **iOS bugs.** Dynamic import of `expo-speech` introduces a race condition producing completely silent coaching with no error. Replacing with a static import resolves this. A second independent bug silently drops the next utterance if `stop()` is called when nothing is playing; checking `isSpeakingAsync()` first prevents this. Both are invisible in any simulator.

B. Local Suggestion Engine

The engine appears immediately when a session ends — before any network call, regardless of connectivity, in under 50 ms. It analyses five dimensions:

Form issue frequency. Every form issue is tracked as a percentage of evaluated frames. Issues present in >10% of frames are reported with exercise-specific biomechanical explanations — not just "knee valgus detected" but "Knee valgus observed in 43% of frames. Inward knee collapse places strain on the ACL and meniscus" [14]. Issues in >60% of frames are escalated to safety warnings regardless of whether they were flagged in real time during the session.

Repetition quality. Tempo consistency is measured via the coefficient of variation across rep durations — the ratio of standard deviation to mean. A high coefficient of variation indicates inconsistent pacing. Fatigue is detected by comparing the mean duration of the first three reps against the

last three: a statistically meaningful increase indicates muscular fatigue within the set, consistent with the tempo analysis methodology of Soro et al. [8].

Depth patterns. Mean joint depth per rep is compared against exercise-specific thresholds. Depth consistency is assessed via standard deviation across reps. Progressive depth loss — where each successive rep reaches a shallower minimum angle — is identified separately from single-rep outliers, because progressive loss indicates fatigue or compensation while outliers may indicate momentary distraction.

Safety escalation. High-risk biomechanical issues — knee valgus, hip sag, knee past toes — undergo a separate safety analysis pass that runs independently of the real-time throttle timers. This ensures safety-critical patterns are always reported in the post-workout summary even if real-time throttling suppressed some in-session warnings.

Cross-session progression. Rep counts and form scores from the current session are compared against all previous sessions stored locally. Improvement patterns are identified across three or more consecutive sessions — a user who has reduced their knee valgus frequency from 45% to 20% to 8% across three sessions receives explicit recognition, because positive reinforcement of improving patterns is as important as correction of persisting ones.

C. RAG-Based Report Generation

After local suggestions are displayed, PoseCoach calls Groq [23] — a hardware inference platform built on custom Language Processing Units (LPUs) designed specifically for high-throughput LLM inference — running Meta Llama 3.3 70B [22] as the underlying language model. The distinction matters: Llama 3.3 70B is the *model* — the neural network created by Meta that understands language and generates the coaching report. Groq is the *platform* it runs on. The same model on typical cloud GPU infrastructure runs at approximately 10–15 tok/s; on Groq's LPU hardware it runs at approximately 276 tok/s [23] — roughly 25× faster, making the difference between a report that appears in 1.3 seconds and one that makes the user wait long enough to put their phone down. The RAG context [18] is session-specific: exercise type, per-rep timing, form issue frequencies, depth data, and the user's full health profile. Local suggestions are included so the LLM builds upon algorithmic analysis rather than replicating it. The model returns structured JSON: biomechanical analysis, 1–4 corrective exercises, coaching tips, an injury risk assessment grounded in form frequencies [15], a next-session focus, and a nutrition tip contextualised to TDEE.

IX. Server Infrastructure and Health Metrics

A. Backend Architecture

Five route groups, all protected by Firebase token middleware except the training endpoint. **/api/workouts** includes server-side ownership validation — the server verifies the requesting UID matches the workout's stored owner UID before returning data. **/api/training-data** is intentionally unprotected during evaluation to minimise data collection friction — a known security limitation requiring authentication before production deployment.

B. Health Metrics

PoseCoach computes a full health profile from six peer-reviewed clinical formulas, feeding directly into every RAG context. **BMR** (Mifflin-St Jeor [24], most accurate for general population [25]): $\text{BMR} = 10m + 6.25h - 5a + s$, where $s = +5$ (male) or -161 (female). $\text{TDEE} = \text{BMR} \times$ Harris-Benedict activity factor (1.2–1.9). **Body fat** (Deurenberg [26]): $1.20 \times \text{BMI} + 0.23 \times a - 10.8 \times \text{sex} - 5.4$. **Max HR** (Tanaka [27], meta-analysis of 351 studies, 18,000+ subjects, substantially more precise than Fox formula): $208 - 0.7 \times a$. **Ideal weight** is computed as an aggregate of four independent formulas — Devine, Robinson, Miller, and Hamwi — producing a range rather than a single target value. Presenting a range reflects the clinical reality that ideal weight varies with frame size and body composition in ways a single formula cannot capture [29]. Daily water intake follows NASEM guidance of 33 ml/kg body weight.

Calorie estimation (MET values from Ainsworth [28]):

$$C = \text{MET} \times m \times t_h \times f_{\text{form}} \times f_{\text{int}} \tag{3}$$

where $f_{\text{form}} \in [1.0, 1.15]$ rewards correct form with up to 15% higher effective MET, and $f_{\text{int}} \in [1.0, 1.2]$ scales with rep cadence.

X. Evaluation

A. Methodology

Evaluation was structured in two phases on supervisor recommendation. **Phase 1** assessed performance across varied real-world conditions: 11 participants performing three sets of eight repetitions of each exercise (squats, push-ups, lunges), producing 792 ground-truth repetitions. Conditions were deliberately varied: different rooms, lighting (bright natural, dim artificial, mixed), camera distances, camera angles, and body types. **Phase 2** was adversarial stress testing: participants were explicitly instructed to attempt to fool the system via deliberate knee valgus, shallow half-reps, rapid exercise transitions, stationary jitter testing, and unusual movement patterns. Ground truth counts were established by manual video review with two independent reviewers confirming any discrepancy.

B. Repetition Counting Accuracy

Table 12: Aggregate counting accuracy, 792 ground-truth repetitions, 11 participants.

Exer.	Done	Count	Miss	FP	Acc.
Squats	264	261	2	2	99.2%
Push-ups	264	263	1	1	99.6%
Lunges	264	263	1	0	99.6%
Total	**792**	**789**	**4**	**3**	**99.1%**

The system counted 789 of 792 repetitions correctly with only 3 false positives. The 4 missed reps all occurred at borderline depth — consistent with the FSM gate's known behaviour. The 3 false positives occurred during unusually rapid squat descents; the cooldown suppressed most but not all at very high descent speed. Critically, *none* of the 3 false positives occurred during stationary jitter testing — they are a distinct failure mode from jitter-induced false counts. The **zero false-positive figure during stationary jitter testing** is therefore the most significant result: the 30°

angular range gate eliminated all jitter-induced false counts across all 11 participants, the primary hazard for any FSM rep counter operating on a noisy mobile pose estimator.

C. Adversarial Stress Testing

Deliberate knee valgus. All participants correctly flagged. The 0.85 ratio threshold was crossed reliably across all body types. **Deliberate shallow half-reps.** 7 false counts recorded across the full adversarial testing session. The gate rejected the majority of attempts, confirming meaningful protection. **Mid-set exercise switching.** Zero misclassifications. The 10-frame majority vote absorbed transition frames cleanly. **Variable conditions.** No degradation under different lighting or camera angles provided the full body remained in frame.

D. Exercise Classification and Form Detection

100% classification accuracy across all 11 sessions, all three exercise types, including adversarial mid-set switching. All deliberately performed form errors correctly detected — knee valgus, forward lean, hip sag, and insufficient depth flagged reliably across body types. No systematic false negatives identified.

E. RAG Report Quality

Post-workout AI reports were successfully generated in all sessions across all three exercise types with no generation failures. Reports were assessed qualitatively as coherent, specific to the session data rather than generic, and containing recommendations that referenced actual session metrics — for example, relating detected knee valgus frequency to specific corrective exercises and linking rep tempo to the participant's body weight and TDEE. The Groq Llama 3.3 70B pipeline returned valid structured JSON in all cases, confirming that the prompt engineering produces reliable structured output at the 276 tok/s throughput. The qualitative nature of this assessment reflects the genuine difficulty of evaluating LLM output rigorously. A rigorous evaluation would require expert physiotherapists to rate each report's clinical accuracy and appropriateness. This represents meaningful future work and a genuine limitation of the current evaluation.

F. Comparison with Related Work

Table 13: Comparison with related exercise analysis systems.

System	Rep.	Class.	Hardware
Soro [8]	±1 (91%)	99.96%	Body-worn IMU
GymCam [7]	±1.7	93.6%	Fixed camera
Ferreira [9]	>92%	—	Fixed camera
RepNet [10]	>95%	N/A	Offline video
Li [12]	>90%	91.3%	Smartphone
PoseCoach	**99.1%**	**100%**	**Smartphone**

PoseCoach achieves competitive or superior accuracy on a consumer smartphone under varied real-world conditions and adversarial testing — without specialised hardware, body-worn sensors, fixed cameras, or offline processing.

G. Requirements Traceability

Table 14 closes the loop between the requirements stated in Section II and the evaluation evidence.

H. Discussion and Limitations

Three limitations must be acknowledged honestly. **Sample size.** Eleven participants cannot support statistically robust population-level claims. A study following the protocol of Stamm and Heimann-Steinert [13] — 20+ participants with documented demographic diversity and clinical ground truth measurement — would be required for publication-grade accuracy claims. **ML pipeline real-world accuracy.** The ML pipeline's real-world accuracy as a standalone measure continues to be measured via the `ComparisonLogger` as real session data accumulates. Sufficient sessions have not yet completed to report definitive figures. **Camera angle constraint.** Performance was robust provided the full body remained in frame. Partial occlusion degrades keypoint detection in ways the visible keypoint count feature (index 12) detects but cannot fully compensate for.

XI. Ethical Considerations

A. Privacy Architecture

No raw camera frames are stored at any point. MoveNet processes each frame and the image is immediately discarded; only derived 14-dimensional feature vectors are retained, and only during explicit recording mode. Personal health data is stored under Firebase Auth and is accessible only to the authenticated, session-owning user.

B. Data Minimisation

Training data upload collects only 14-dimensional feature vectors and rule-based labels — never raw images, video, or identifiable biometric data. Feature vectors cannot be reverse-engineered to reconstruct a user's appearance.

C. Medical Disclaimer

PoseCoach is not a medical device. RAG-generated injury risk assessments are not clinically validated and must not substitute professional physiotherapy or medical advice. Users with known musculoskeletal conditions should consult a healthcare provider before relying on automated form feedback. This disclaimer is presented within the application.

D. Security Limitation and Remediation Plan

The `/api/training-data` endpoint is intentionally unprotected during evaluation to minimise data collection friction. Before any production deployment, this endpoint must be protected by Firebase token verification identical to all other routes, and explicit informed consent must be obtained before any participant's data is uploaded. In addition, the `/api/training-data` endpoint should be rate-limited and require server-side validation that uploaded feature vectors conform to the expected 14-dimensional schema to prevent data poisoning attacks on the training pipeline.

E. Anthropometric Coverage

The synthetic training data models diversity through ±8% height and ±6% limb proportion variation but does not capture full global anthropometric diversity — a limitation the continuous learning system progressively addresses as real-world usage accumulates across diverse users.

Table 14: Requirements traceability: all functional and non-functional requirements met.

ID	Description	How Requirement is Met	Status
FR1	Detect 17 joints ≥2 FPS	MoveNet Lightning at 2–5 FPS on iOS (Table 15)	✓ Met
FR2	Auto-classify exercise type	RuleBasedClassifier + ML Classifier; 100% accuracy (§X-D)	✓ Met
FR3	Count reps: <2% miss, 0 FP	FSM + 30° gate; 99.1% acc., 0 FP jitter test (Table 12)	✓ Met
FR4	≥3 form issues per exercise	10 issues across 3 exercises (Table 5); all detected	✓ Met
FR5	Spoken corrections, no flow break	TTSManager 5-level priority, per-issue throttle (Table 11)	✓ Met
FR6	Personalised post-workout report	Local engine <50 ms + RAG Llama 3.3 70B ≈1.3 s (§VIII-B,C)	✓ Met
FR7	Fully offline during workout	All inference on-device; no network calls during session (§IV-A)	✓ Met
FR8	Persist history & health metrics	AsyncStorage + Cloud Firestore sync; 6 clinical formulas (§IX-B)	✓ Met
NFR1	Frame latency <500 ms	2–5 FPS pipeline; full cycle <500 ms on iOS (Table 15)	✓ Met
NFR2	No raw frames transmitted	Frames discarded after inference; only 14-dim vectors stored (§XI-A)	✓ Met
NFR3	ML bundle <150 KB	Float32Array embedding: 127 KB total (Table 15)	✓ Met
NFR4	No crashes on iOS/Expo Go	Zero crashes across 792-rep evaluation (§X-A)	✓ Met
NFR5	Biometrics auth-gated	Firebase Auth; server-side UID ownership validation (§IX-A)	✓ Met

XII. Implementation Summary

PoseCoach comprises approximately 80 source files organised across three tiers. The mobile client contains 14 screens, over 20 reusable components, 4 Redux slices, and 12 service modules including the `AnalysisOrchestrator`, `TTSManager`, `featureExtractor`, `mlPipeline`, `poseStorage`, and `ragReportService`. The ML training pipeline consists of a production data generator (`generateProductionData.js`), a training engine (`trainModels.js`), and a weight exporter (`exportWeights.js`). The backend provides 5 route groups with Firebase Admin token verification. All five ML models total 13,707 parameters and load in under 100 ms via the Float32Array embedding strategy. Table 15 summarises key performance metrics measured on physical iOS hardware under Expo Go.

Table 15: Key system performance metrics measured on iOS under Expo Go.

Metric	Value
Total source files	≈80
ML model load time	<100 ms
ML bundle overhead	127 KB
Total ML parameters	13,707
Suggestion engine latency	<50 ms (offline)
RAG report latency	≈1.3 s
AsyncStorage session write	<30 ms
On-device fine-tune (500 fr.)	≈8 s
Frame rate (Expo Go, iOS)	2–5 FPS

XIII. Conclusion

A. Summary of Achievements

PoseCoach demonstrates that real-time exercise recognition, repetition counting, biomechanical form feedback, and personalised coaching can be delivered through a consumer smartphone with no specialised hardware, no internet connectivity during a workout, and no native code compilation. Evaluated across 11 participants and 792 repetitions under varied real-world conditions and adversarial testing, the system achieves 99.1% counting accuracy, 100% classification accuracy, zero false positives during stationary jitter testing, and correct detection of all deliberately performed form errors.

B. Engineering Contributions

Five engineering contributions generalise beyond this application. The **Float32Array embedding strategy** solves the `bundleResourceIO` constraint for any developer deploying TF.js models in a React Native managed environment. The 30° **angular range gate** is a general solution to FSM-based repetition counting in the presence of noisy mobile pose estimation. The **ComparisonLogger pattern** — running competing pipelines simultaneously and logging per-frame agreement as a production instrument — generalises to any deployed ML system where models are trained on synthetic data and must improve from real usage. The **EXIF normalisation fix** addresses a class of device-orientation bugs in mobile computer vision pipelines that manifest only on specific devices in specific orientations. The **independent per-issue TTS throttle timer pattern** is a general solution to multi-issue real-time voice feedback where a shared throttle renders coaching functionally useless.

C. Future Work

The highest-priority improvement is closing the synthetic-to-real domain gap through real data accumulation and periodic server-side retraining. Migrating from Expo Go to a development build would enable native TFLite inference at 15–30 FPS. Expanding the exercise library to deadlifts, overhead press, and burpees would broaden utility substantially. A rigorous study with 20+ participants following clinical validation methodology [13] would provide statistically robust accuracy estimates. The accessibility gap in fitness coaching is real and consequential. PoseCoach demonstrates it is technically closeable with the hardware people already carry, the open-source models already available, and the engineering approaches described in this report.

References

[1] R. Votel and C. Li, "Next-generation pose detection with MoveNet and TensorFlow.js," *Google TensorFlow Blog*, May 2021. `https://blog.tensorflow.org/2021/05/next-generation-pose-detection-with-movenet-and-tensorflowjs.html`

[2] A. Toshev and C. Szegedy, "DeepPose: Human pose estimation via deep neural networks," in *Proc. IEEE CVPR*, 2014. `https://doi.org/10.1109/CVPR.2014.214`

[3] Z. Cao, T. Simon, S.-E. Wei, and Y. Sheikh, "Realtime multi-person 2D pose estimation using part affinity fields," in *Proc. IEEE CVPR*, 2017. `https://doi.org/10.48550/arXiv.1611.08050`

[4] K. Sun, B. Xiao, D. Liu, and J. Wang, "Deep high-resolution representation learning for visual recognition," in *Proc. IEEE/CVF CVPR*, 2019. `https://doi.org/10.1109/CVPR.2019.00584`

[5] T.-Y. Lin et al., "Microsoft COCO: Common objects in context," in *Proc. ECCV*, 2014. `https://doi.org/10.1007/978-3-319-10602-1_48`

[6] V. Bazarevsky et al., "BlazePose: On-device real-time body pose tracking," *arXiv:2006.10204*, 2020. `https://doi.org/10.`

48550/arXiv.2006.10204

[7] R. Khurana et al., "GymCam: Detecting, recognizing and tracking simultaneous exercises," *Proc. ACM Interact. Mob. Wearable Ubiquitous Technol.*, vol. 2, no. 4, 2018. https://doi.org/10.1145/3287063

[8] A. Soro, G. Brunner, S. Tanner, and R. Wattenhofer, "Recognition and repetition counting for complex physical exercises with deep learning," *Sensors*, vol. 19, no. 3, p. 714, 2019. https://doi.org/10.3390/s19030714

[9] B. Ferreira and J. Ferreira, "Exploring workout repetition counting through deep learning," in *Proc. ICIAR*, 2020. https://doi.org/10.1007/978-3-030-50347-5_1

[10] D. Dwibedi et al., "Counting out time: Class agnostic video repetition counting in the wild," in *Proc. IEEE/CVF CVPR*, 2020. https://openaccess.thecvf.com/content_CVPR_2020/html/Dwibedi_Counting_Out_Time_Class_Agnostic_Video_Repetition_Counting_in_the_CVPR_2020_paper.html

[11] Z. Yao, X. Cheng, and Y. Zou, "PoseRAC: Pose saliency transformer for repetitive action counting," *arXiv:2303.08450*, 2023. https://doi.org/10.48550/arXiv.2303.08450

[12] Z. Li et al., "Real-time fitness exercise classification and counting from video streams," *arXiv:2411.11548*, 2024. https://doi.org/10.48550/arXiv.2411.11548

[13] O. Stamm and A. Heimann-Steinert, "Accuracy of monocular 2D pose estimation compared with a reference standard for kinematic multiview analysis," *JMIR mHealth uHealth*, 8(12):e19608, 2020. https://doi.org/10.2196/19608

[14] C. Balsalobre-Fernández et al., "A beta version of an application based on computer vision for the assessment of knee valgus angle: A validity and reliability study," *Sensors*, vol. 23, no. 3, p. 1432, 2023. https://www.mdpi.com/2227-9032/11/9/1258

[15] C. Mennella, U. Maniscalco, G. De Pietro, and M. Esposito, "A deep learning system to monitor and assess rehabilitation exercises in home-based remote and unsupervised conditions," *Comput. Biol. Med.*, 166:107485, 2023. https://www.sciencedirect.com/science/article/pii/S0010482523009502?via%3Dihub

[16] A. G. Howard et al., "MobileNets: Efficient CNNs for mobile vision applications," *arXiv:1704.04861*, 2017. https://doi.org/10.48550/arXiv.1704.04861

[17] J. Kirkpatrick et al., "Overcoming catastrophic forgetting in neural networks," *Proc. Natl. Acad. Sci.*, vol. 114, pp. 3521–3526, 2017. https://doi.org/10.1073/pnas.1611835114

[18] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in NeurIPS*, 2020. https://doi.org/10.48550/arXiv.2005.11401

[19] D. Smilkov et al., "TensorFlow.js: Machine learning for the web and beyond," in *Proc. SysML*, 2019. https://doi.org/10.48550/arXiv.1901.05350

[20] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. ICLR*, 2015. https://doi.org/10.48550/arXiv.1412.6980

[21] L. Prechelt, "Early stopping — but when?," in *Neural Networks: Tricks of the Trade*, Springer, 1998. https://doi.org/10.1007/3-540-49430-8_3

[22] Meta AI, "The Llama 3 herd of models," *arXiv:2407.21783*, 2024. https://doi.org/10.48550/arXiv.2407.21783

[23] Groq Inc., "Groq LPU inference engine benchmarks," Technical Report, groq.com, 2024. https://groq.com/newsroom/groq-lpu-inference-engine-leads-in-first-independent-llm-benchmark

[24] M. D. Mifflin et al., "A new predictive equation for resting energy expenditure," *Am. J. Clin. Nutr.*, vol. 51, pp. 241–247, 1990. https://doi.org/10.1093/ajcn/51.2.241

[25] D. Frankenfield, L. Roth-Yousey, and C. Compher, "Comparison of predictive equations for resting metabolic rate," *J. Am. Diet. Assoc.*, vol. 105, no. 5, pp. 775–789, 2005. https://doi.org/10.1016/j.jada.2005.02.005

[26] P. Deurenberg, J. A. Weststrate, and J. C. Seidell, "Body mass index as a measure of body fatness," *Br. J. Nutr.*, vol. 65, pp. 105–114, 1991. https://doi.org/10.1079/BJN19910073

[27] H. Tanaka, K. D. Monahan, and D. R. Seals, "Age-predicted maximal heart rate revisited," *J. Am. Coll. Cardiol.*, vol. 37, pp. 153–156, 2001. https://doi.org/10.1016/S0735-1097(00)01054-8

[28] B. E. Ainsworth et al., "2011 Compendium of physical activities," *Med. Sci. Sports Exerc.*, vol. 43, pp. 1575–1581, 2011. https://doi.org/10.1249/MSS.0b013e31821ece12

[29] World Health Organization, "WHO guidelines on physical activity and sedentary behaviour," WHO, Geneva, 2020. https://www.who.int/publications/i/item/9789240015128